InsaCloud

Script de soutenance

15 minutes · trois intervenants · démonstration en direct

À chaque geste de la démonstration, celui qui parle nomme **l'outil employé** et **la fonctionnalité du cahier des charges** qu'il réalise. La prof doit pouvoir cocher sa grille sans avoir à nous le demander.

Toutes les commandes de ce document ont été exécutées sur la plateforme. Toutes les captures d'écran viennent de l'application telle qu'elle tourne, et les sorties reproduites sont les vraies.

Projet 3 — *Outils de déploiement de plateformes*
INSA Centre-Val de Loire, STI 4A FISE, 2026/2027 — Dr. Nadia Fettah

13 octobre 2026

Table des matières

Avant d'entrer dans la salle 1

1 Personne 1 — Le problème et l'infrastructure 3

2 Personne 2 — Le parcours utilisateur 6

3 Personne 3 — Scénarios, sécurité, chaîne 15

4 Personne 1 — Conformité, ajouts, conclusion 19

5 Les questions probables 21

6 Si quelque chose casse en direct 23

Avant d'entrer dans la salle

À faire **vingt minutes** avant, pas deux.

```
colima start --cpu 4 --memory 6 --disk 40
cd ~/Developer/insacloud && docker compose -p insacloud up -d
```

Vérifier les trois conteneurs. Les trois doivent être `healthy`. S'il en manque un, ne pas entrer : relancer.

```
docker compose -p insacloud ps
```

```
NAME                STATUS
insacloud-db        Up 24 hours (healthy)
insacloud-faucheur  Up 7 hours (healthy)
insacloud-web       Up 7 hours (healthy)
```

Vérifier les six images. Sans elles, le mode bureau n'apparaîtra pas dans le formulaire — et ce n'est pas un bug, c'est voulu : le catalogue ne propose pas ce qu'il ne peut pas livrer.

```
docker images --filter 'reference=insacloud_*'
```

```
insacloud_alpine  118MB
insacloud_alpine_desktop  1.35GB
insacloud_debian  303MB
insacloud_debian_desktop  1.49GB
insacloud_ubuntu  242MB
insacloud_ubuntu_desktop  1.33GB
insacloud_web  414MB
```

Préparer les onglets , dans cet ordre :

1. `http://127.0.0.1:8088` — connecté, tableau de bord **vide**
2. `https://github.com/aziz3r/insacloud/actions` — la chaîne au vert
3. un terminal plein écran dans `~/Developer/insacloud`
4. le `README.md` du dépôt, prêt à montrer les schémas

Si le cluster Vagrant est monté : un quatrième onglet sur `https://192.168.56.10` et un second terminal prêt pour `vagrant halt worker1`.

À retenir

Personne ne tape une commande qu'il n'a pas répétée. Chaque commande de ce document a été exécutée au moins une fois avant la soutenance.

Répartition

Qui	Quand	Rôle
Personne 1	0 :00 – 3 :30	Le problème, l'architecture, l'infrastructure
Personne 2	3 :30 – 9 :00	Le parcours utilisateur, en direct
Personne 3	9 :00 – 13 :30	Les scénarios de l'énoncé, la sécurité, la chaîne
Personne 1	13 :30 – 15 :00	Conformité, ajouts, conclusion

CHAPITRE 1

Personne 1 — Le problème et l'infrastructure

0 :00 — Ouvrir (30 s)

InsaCloud est un mini-fournisseur de cloud. Un utilisateur s'inscrit, choisit une distribution Linux et une durée, et obtient en quinze secondes une machine à lui, avec un accès root. À l'échéance, elle disparaît toute seule.

Derrière ce clic il y a quatre machines, un ordonnanceur, un démon de nettoyage, un coffre à secrets et onze contrôles automatiques. C'est ce que nous allons dérouler.

0 :30 — Le choix fondateur (45 s)

À faire — montrer

Le schéma d'architecture, dans le `README.md` du dépôt.

*Première décision : des **conteneurs**, pas des machines virtuelles. Une VM démarre en trente secondes et réserve des centaines de mégaoctets rien que pour son noyau. Un conteneur réutilise le noyau de l'hôte : il démarre en moins d'une seconde. Pour une location **à la minute**, c'est décisif — et l'énoncé le tranche lui-même : « des conteneurs dans notre cas ».*

*Deuxième décision : séparer **qui décide** de **qui exécute**. Le contrôleur porte le site, la base et l'ordonnanceur. Il n'exécute aucune machine louée et n'a même pas de démon Docker, seulement le client. Les trois workers portent Docker et les conteneurs. Un utilisateur qui s'échapperait de son conteneur tomberait sur un worker : pas de base, pas de clés, pas de code.*

1 :15 — Comment le contrôleur pilote les workers (45 s)

À faire — taper

```
docker -H ssh://insacloud@worker1 ps
```

En mode compose, montrer plutôt `2_webapp/workers.py`, fonction `docker_command`.

On aurait pu exposer l'API Docker sur le réseau, port 2375. C'est simple et c'est une très mauvaise idée : qui parle au démon Docker d'une machine en est root. Une erreur de pare-feu et toute la plate-forme tombe.

*On passe donc par **SSH**, que le client Docker sait faire nativement. Aucun port Docker n'est ouvert, l'authentification est celle de SSH par clé, et le compte de service n'a pas de mot de passe.*

→ **Fonctionnalité 4 (conteneurisation)** · **Fonctionnalité 14 (sécurité)**

2 :00 — L'infrastructure comme code (1 min)

À faire — montrer

Le `Vagrantfile`, puis `3_ansible/site.yml`.

*Rien n'est installé à la main. **Vagrant** décrit les quatre machines virtuelles — un contrôleur en .10, trois workers en .11, .12, .13 — avec leur mémoire, leurs cœurs, leur réseau privé. Le fichier gère quatre providers, dont VMware, parce que VirtualBox ne tourne pas sur une puce Apple.*

***Ansible** les configure : neuf rôles, quatre playbooks. Pare-feu, clé de pilotage, Docker, certificats TLS, images, agent, proxy inverse, application.*

*Et surtout : le playbook est **idempotent**. Le relancer sur un serveur déjà configuré ne change rien. La preuve se lit dans le compte rendu : au second passage, `changed=0` sur les quatre nœuds. Ce n'est pas « ça ne fait rien la deuxième fois » : c'est « il vérifie l'état et n'agit que s'il diffère ».*

À faire — taper, si le cluster est monté

```
cd 3_ansible
ansible-playbook -i inventaire_vagrant.py site.yml --vault-
```

```
    ↪ password-file .vault_pass
```

→ **Fonctionnalité 5 (Vagrant) · Fonctionnalité 6 (Ansible)**

3 :00 — Les secrets (30 s)

À faire — montrer

`3_ansible/group_vars/insacloud/vault.yml` — illisible à l'écran, c'est le but.

*Les secrets vivent dans un **Ansible Vault**, chiffré en AES-256. Ce fichier est versionné dans Git tel quel : sans la phrase de passe, il ne dit rien. Il contient la clé de session Flask, la clé du coffre, et le jeton des agents — trois clés distinctes, jamais la même.*

→ **Fonctionnalité 6 · Fonctionnalité 14**

Transition : « [Prénom 2] va vous montrer ce que ça donne du côté de l'utilisateur. »

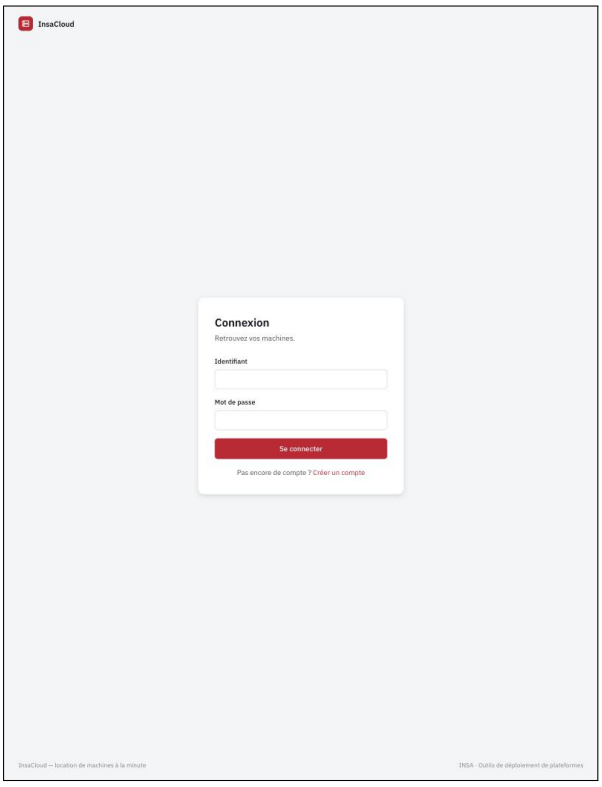
CHAPITRE 2

Personne 2 — Le parcours utilisateur

Piège — La règle de cette partie

Tout se fait **en cliquant**, pas en racontant. On parle *pendant* que ça charge — c'est là que se gagne la note.

3 :30 — Inscription et connexion (40 s)



À faire — faire

Créer un compte en direct, puis se connecter.

*L'application est en **Flask**, servie par **Gunicorn** avec quatre processus ouvriers, derrière **nginx** qui termine le TLS. Flask seul traite une requête à la fois et le dit lui-même : à ne pas utiliser en production.*

*La politique de mot de passe est vérifiée **côté serveur**. Un contrôle fait seulement dans le navigateur se contourne en coupant JavaScript. Les mots de passe de compte sont **hachés** en pbkdf2-sha256 : opération à sens unique, on ne peut pas les retrouver. Et cinq échecs de connexion verrouillent le compte.*

→ **Fonctionnalité 2 (API Web Flask)** · **Fonctionnalité 14**

4 :10 — Le catalogue (30 s)

*Trois distributions — Ubuntu, Debian, Alpine — en deux modes, soit **six images** construites depuis **quatre Dockerfiles**. Le mode bureau s'active par un simple `--build-arg DESKTOP=1`.*

*Le catalogue n'est pas écrit en dur : il est **en base**, et l'application interroge Docker avant d'afficher un choix. Si une image n'est pas construite, la distribution n'apparaît pas — on ne propose pas ce qu'on ne peut pas livrer.*

→ **Fonctionnalité 3 (base de données)** · **Fonctionnalité 4**

4 :40 — Louer (50 s)

À faire — faire

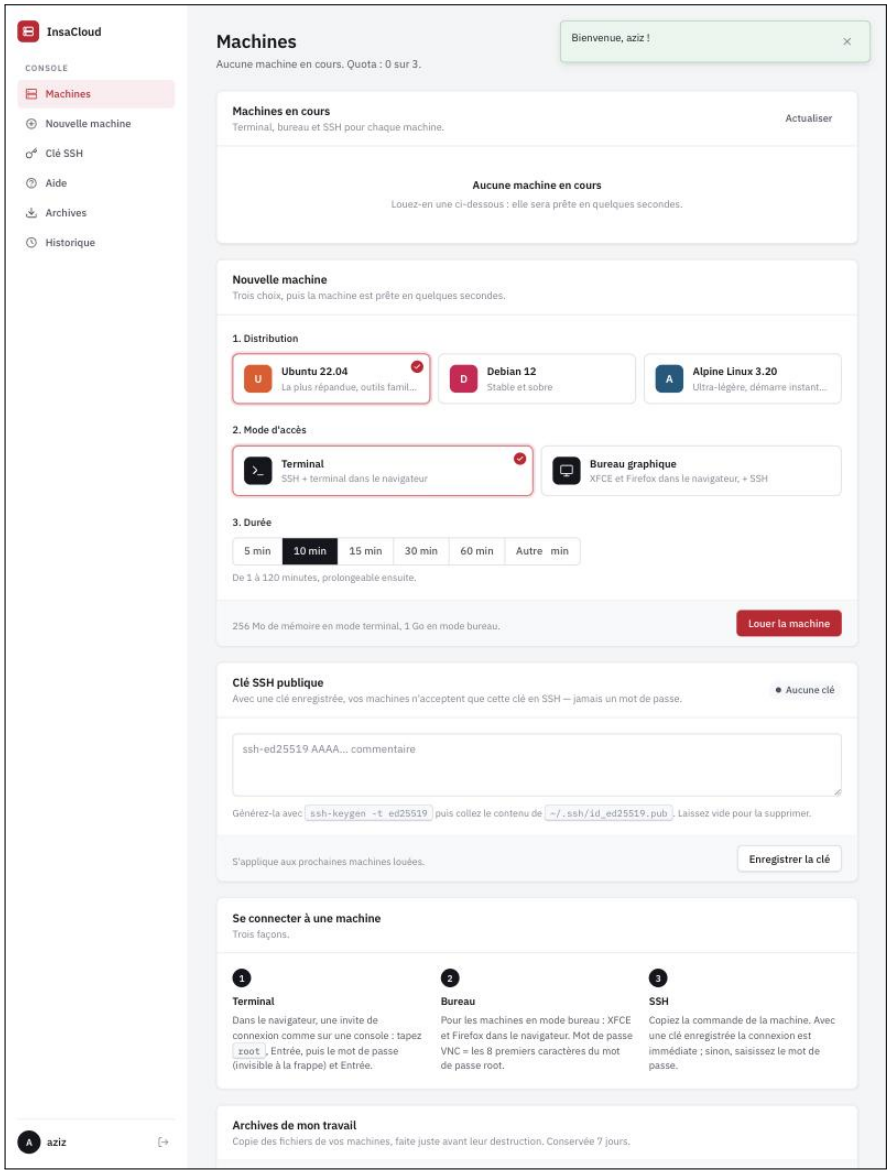
Ubuntu · Bureau graphique · 10 minutes · **Louer la machine**. Parler pendant les quinze secondes de démarrage.

Ce qui part côté serveur, c'est exactement la commande imposée par l'énoncé :

```
docker run -d --restart=always -p <port>:22 <image>
```

*On la pilote par `subprocess`, pas par un SDK — comme ça chaque option se lit telle quelle dans le code. Et jamais avec `shell=True` : les arguments sont passés en **liste**, donc aucun interpréteur ne relit la chaîne. Sinon un utilisateur nommerait sa machine `x; rm -rf /`.*

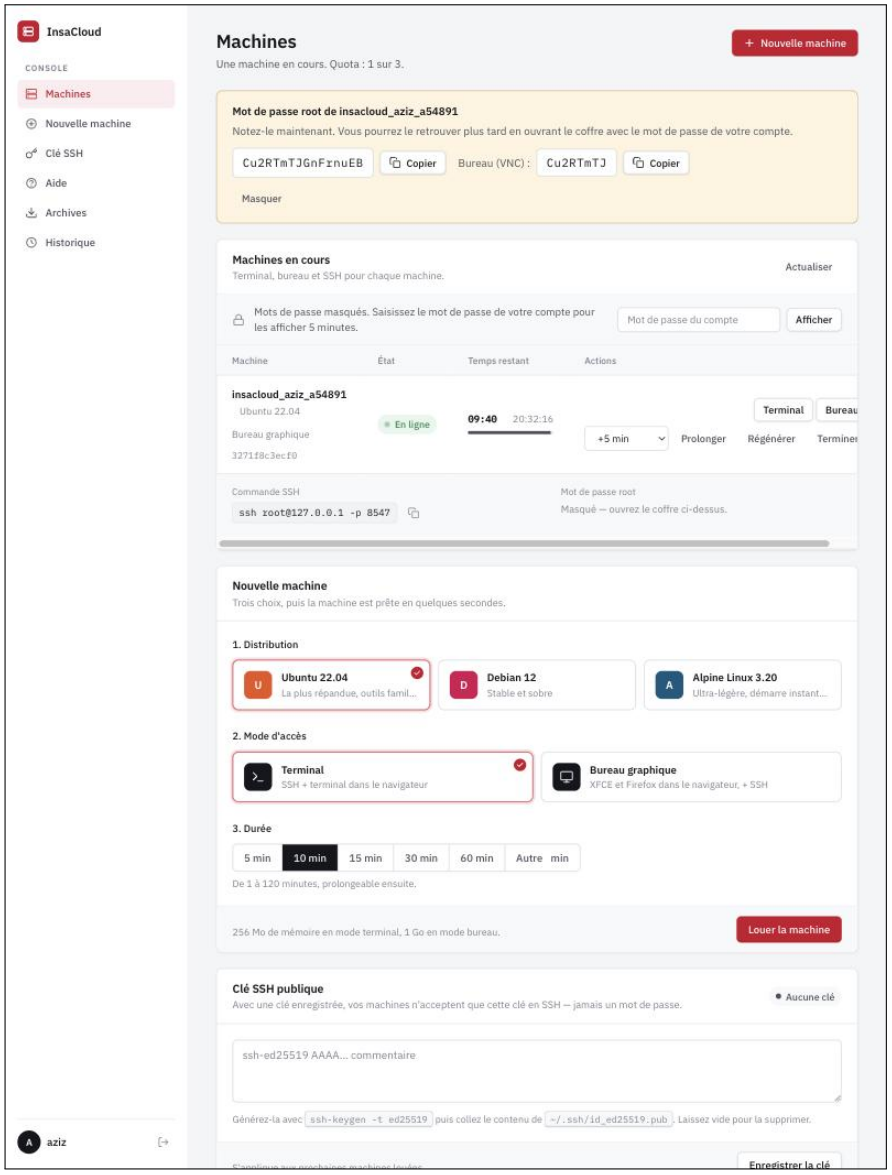
*S'y ajoutent le durcissement : `--cap-drop ALL` puis seulement les dix capacités dont `sshd` a besoin, `no-new-privileges`, une limite de processus, de CPU, et la mémoire — **réservation et limite**, comme le demande le cours.*



> *Le port vient de la plage 8000–9000, et son unicité est garantie par un* ***index unique partiel*** *en base, limité aux machines actives. Deux requêtes simultanées ne peuvent pas prendre le même port.*

→ Fonctionnalité 9 (service de location) · Fonctionnalité 10 (déploiement des instances) · Fonctionnalité 8 (ordonnanceur)

5 :30 — Le secret montré une fois (40 s)

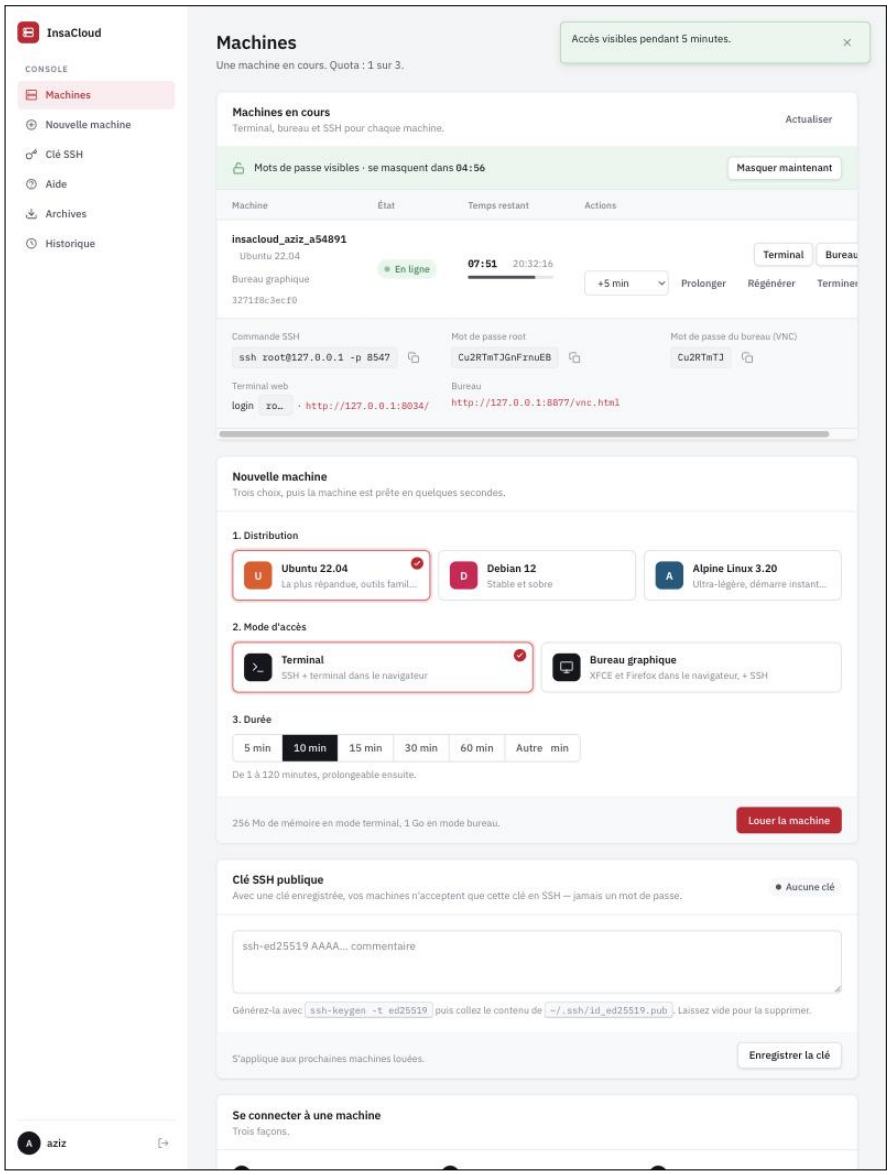


Le mot de passe root est tiré au sort, seize caractères, environ 90 bits. Il est affiché ***une seule fois****, puis chiffré avec* ***Fernet*** *avant d'être écrit en base — clé distincte de celle des sessions.*

Une erreur qu'on a commise au début : on l'avait mis dans le cookie de session. Or une session Flask est ***signée, pas chiffrée*** *: n'importe qui pouvait la décoder en base64. On ne stocke plus que l'identifiant de la machine, et le secret est relu en base.*

À faire — faire

Recharger la page : le mot de passe a disparu. Puis ressaisir le mot de passe du compte — le coffre s'ouvre cinq minutes.



→ **Fonctionnalité 14 (protection des secrets)**

6 :10 — Les trois accès (1 min 20)

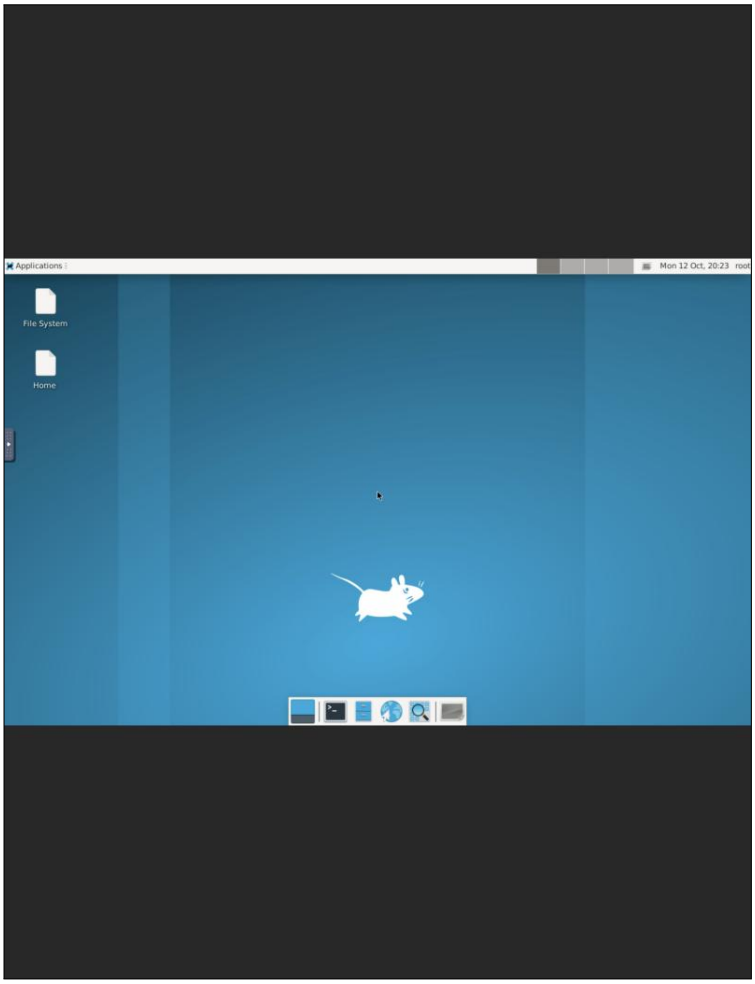
À faire — faire, dans cet ordre

1. **SSH** — copier la commande affichée, se connecter depuis le terminal.
2. **Terminal web** — ouvrir le lien.
3. **Bureau graphique** — ouvrir, saisir le mot de passe VNC.

***SSH**, c'est l'accès demandé par l'énoncé. Par mot de passe, ou par clé si l'utilisateur a enregistré sa clé publique — dans ce cas la machine n'accepte **plus que** la clé.*



*Le **terminal web**, c'est `ttyd`. Attention : le port ne donne pas un shell ouvert, il lance `login`. Il faut s'authentifier.*



*Le **bureau**, c'est `Xvnc`, `XFCE` et `noVNC`. Cinq processus cohabitent dans ce conteneur : sshd, ttyd, le serveur d'affichage, le bureau et la passerelle web. Or un conteneur n'exécute qu'un processus — d'où **supervisord** en PID 1, qui les démarre et relance celui qui meurt.*

→ **Fonctionnalité 11 (accès SSH)** · **Fonctionnalité 4**

7 :30 — L'expiration (1 min)

À faire — faire

Louer une machine **1 minute**. Pendant qu'elle vit :

```
docker logs -f insacloud-faucheur
```

*Personne ne viendra cliquer sur « supprimer » à trois heures du matin. Un service séparé, qu'on appelle le **Faucheur**, lit la base toutes les dix secondes et détruit ce qui a expiré.*

*Il fait aussi de la **réconciliation** : la base dit ce qui devrait exister, Docker ce qui existe vraiment. Un conteneur orphelin est supprimé, une instance sans conteneur passe à « arrêtée ». Avec une règle essentielle : un nœud injoignable est **ignoré**, jamais interprété comme vide — sinon on marquerait arrêtées des machines qui tournent très bien.*

*Et il reconnaît ses machines à une **étiquette Docker**, jamais à leur nom. Au début il les reconnaissait par préfixe — et il a fini par détruire les conteneurs de la plateforme elle-même.*

À faire — montrer

La machine disparaît du tableau de bord, le quota repasse à zéro. Puis les étiquettes qui lui donnent son identité :

```
{
    "com.docker.compose.image.builder": "classic",
    "description": "Machine Ubuntu louable (SSH + terminal web, option bureau
        ↪ XFCE)",
    "insacloud.cluster": "compose",
    "insacloud.role": "rented-machine",
    "maintainer": "InsaCloud - Projet INSA STI 4A",
    "org.opencontainers.image.version": "22.04"
}
```

→ **Fonctionnalité 12 (gestion de l'expiration)**

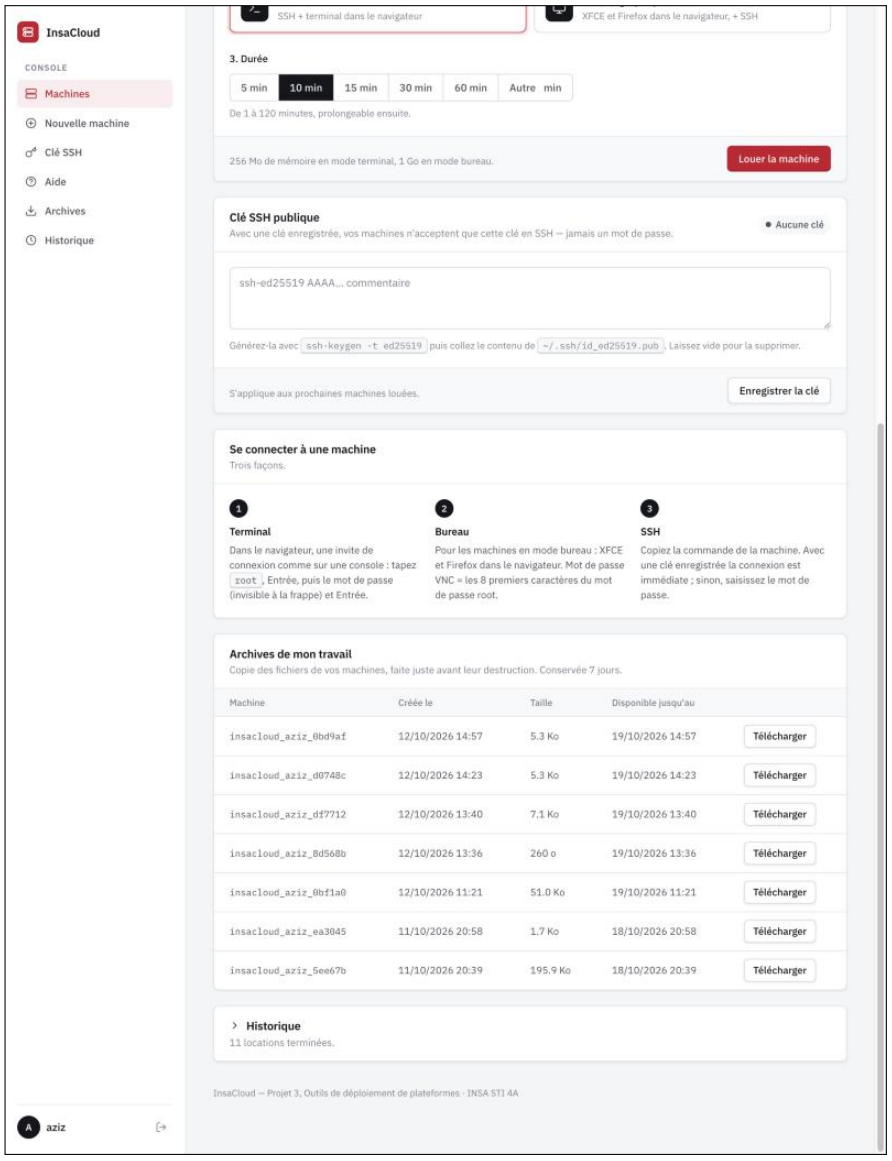
8 :30 — L'ajout : la sauvegarde du travail (30 s)

Ça, l'énoncé ne le demande pas. Détruire un conteneur détruit sa couche inscriptible : l'utilisateur perd tout. Juste avant le `docker rm -f`, on fait un `tar` de `/root` et `/home` qu'on lit au fil de l'eau, et l'utilisateur le télécharge pendant sept jours.

*Avec une règle qui prime sur tout : l'archivage **ne bloque jamais** la destruction. Sinon une machine expirée sur un nœud en panne resterait en vie indéfiniment.*

→ **Au-delà de l'énoncé**

Transition : « [Prénom 3] va vous montrer ce qui se passe quand ça casse. »


InsaCloud
CONSOLE
Machines
Nouvelle machine
Clé SSH
Aide
Archives
Historique
aziz
SSH + terminal dans le navigateur
XFCE et Firefox dans le navigateur, + SSH
3. Durée
5 min
10 min
15 min
30 min
60 min
Autre min
De 1 à 120 minutes, prolongeable ensuite.
256 Mo de mémoire en mode terminal, 1 Go en mode bureau.
Louer la machine
Clé SSH publique
Avec une clé enregistrée, vos machines n'acceptent que cette clé en SSH — jamais un mot de passe.
Aucune clé
ssh-ed25519 AAAA... commentaire
Générez-la avec ssh-keygen -t ed25519 puis collez le contenu de ~/.ssh/id_ed25519.pub. Laissez vide pour la supprimer.
S'applique aux prochaines machines louées.
Enregistrer la clé
Se connecter à une machine
Trois façons.
1
Terminal
Dans le navigateur, une invite de connexion comme sur une console : tapez root, Entrée, puis le mot de passe (invisible à la frappe) et Entrée.
2
Bureau
Pour les machines en mode bureau : XFCE et Firefox dans le navigateur. Mot de passe VNC = les 8 premiers caractères du mot de passe root.
3
SSH
Copiez la commande de la machine. Avec une clé enregistrée la connexion est immédiate ; sinon, saisissez le mot de passe.
Archives de mon travail
Copie des fichiers de vos machines, faite juste avant leur destruction. Conservée 7 jours.
Machine
Créée le
Taille
Disponible jusqu'au
insacloud_aziz_0bd9af
12/10/2026 14:57
5.3 Ko
19/10/2026 14:57
Télécharger
insacloud_aziz_d0748c
12/10/2026 14:23
5.3 Ko
19/10/2026 14:23
Télécharger
insacloud_aziz_df7712
12/10/2026 13:40
7.1 Ko
19/10/2026 13:40
Télécharger
insacloud_aziz_8d568b
12/10/2026 13:36
260 o
19/10/2026 13:36
Télécharger
insacloud_aziz_0bf1a0
12/10/2026 11:21
51.0 Ko
19/10/2026 11:21
Télécharger
insacloud_aziz_ea3045
11/10/2026 20:58
1.7 Ko
18/10/2026 20:58
Télécharger
insacloud_aziz_5ee67b
11/10/2026 20:39
195.9 Ko
18/10/2026 20:39
Télécharger
Historique
11 locations terminées.
InsaCloud — Projet 3, Outils de déploiement de plateformes · INSA ST1 4A

CHAPITRE 3

Personne 3 — Scénarios, sécurité, chaîne

9 :00 — Le parc qui se gère seul (50 s)

À faire — taper

```
curl -s -H 'Accept: application/json' http://127.0.0.1:8088/
    ↪ workers | jq
```

Le contrôleur ne contient ***aucune adresse de worker****. Chaque nœud porte un petit* ***agent****, écrit en bibliothèque standard seulement — pas de dépendance à installer sur les workers. Au démarrage il se présente, puis il signale sa présence toutes les trente secondes.*

Trois états : `AVAILABLE`*,* `BUSY` *quand le nœud est plein,* `OFFLINE` *après quatre-vingt-dix secondes de silence. La distinction compte : un nœud saturé est en bonne santé, il ne faut surtout pas migrer ses machines.*

Conséquence concrète : pour ajouter un quatrième worker, on l'allume. C'est tout.

→ Fonctionnalité 7 (Worker Agent) · Fonctionnalité 8 (ordonnanceur)

9 :50 — Le Test 6 de l'énoncé : la panne (1 min 40)

Piège — Deux chemins possibles

Si le cluster Vagrant est monté, faire la manipulation en direct. **Sinon**, lire le repli en fin de section — et le dire franchement, plutôt que de laisser croire.

À faire — faire

```
vagrant halt worker1
vagrant ssh controller -c 'journalctl -u insacloud-faucheur -f'
```

L'énoncé décrit précisément ce test : on éteint `worker1`, et on vérifie que `worker1` passe `OFFLINE` et que l'instance repart sur `worker2`.

L'agent cesse de battre. Au bout de quatre-vingt-dix secondes le contrôleur déclare le nœud hors ligne, et recrée ses machines sur un nœud disponible.

*Ce qui compte, c'est ce que le code **ne** touche **pas** : la table des locations. C'est la décision qui structure toute la base — on a séparé `Instance`, qui décrit un conteneur, de `Rental`, qui décrit l'engagement envers l'utilisateur. Si on avait fondu les deux en une table, plus court à écrire, la reprise serait **impossible** : on ne pourrait pas recréer la machine ailleurs sans toucher à la durée réservée.*

L'utilisateur garde donc sa location et sa durée ; il reçoit un nouveau port et un nouveau mot de passe. Et un compteur `migrations` passe à 1 : c'est la preuve observable, celle que vérifie notre test d'intégration.

*Une limite qu'on assume : la reprise ne rend **pas les fichiers**. Ils vivaient dans la couche inscriptible, sur un nœud devenu injoignable. Sans stockage partagé, seul le service repart. Un volume répliqué changerait la nature du projet.*

Piège — Repli si le cluster n'est pas disponible

« La reprise ne peut pas tourner sur ce poste : le parc demande onze gigaoctets de mémoire, la machine en a huit. Le scénario a été mesuré sur un cluster réel à quatre nœuds — c'est dans le dossier technique : `worker1` éteint, `OFFLINE` à quatre-vingt-dix secondes, machine recréée sur `worker2`, location intacte, `migrations=1`, et `worker1` de retour en moins de dix secondes. »

→ Fonctionnalité 13 (haute disponibilité) · Test 6 de l'énoncé

11 :30 — Les six scénarios d'intégration (40 s)

À faire — taper

```
cd 2_webapp && ./tests/integration.sh http://127.0.0.1:8088
```

L'énoncé liste six tests. Ils sont scriptés et rejouables : infrastructure, configuration Ansible, application, location avec connexion SSH réelle, expiration, et panne d'un nœud.

*À côté, **162 tests unitaires** en vingt secondes. Et ils ne lancent*

> *aucun conteneur : `run_docker` est remplacé par une fonction espion. On vérifie donc que la commande imposée part bien telle quelle, sur un agent d'intégration qui n'a même pas Docker.*

```
.....................................................................  [ 97%]
...                                                                    [100%]
147 passed in 15.73s
```

→ **Fonctionnalité 16 (tests d'intégration)**

12 :10 — La sécurité DevSecOps (40 s)

> **À faire — montrer**
>
> L'onglet GitHub Actions, la liste des travaux. Puis, si on a le temps, le durcissement lu sur une machine réelle :

```
RestartPolicy    always
Memory           1024 Mio    reservation 512 Mio
NanoCpus         1.0 coeur
PidsLimit        512
CapDrop          ['ALL']
CapAdd           10 capacites rendues
SecurityOpt      ['no-new-privileges:true']
PortBindings     {'22/tcp': '8547', '6080/tcp': '8877', '7681/tcp': '8034'}
```

> *Le « Sec » de DevSecOps, c'est que la sécurité est un contrôle **automatique**, pas un audit de fin de projet. Six familles :*
>
> ***bandit** cherche les motifs dangereux dans le code : injection, secret en dur, appel système construit depuis une entrée utilisateur. **pip-audit** confronte nos dépendances aux bases de CVE. **gitleaks** cherche les secrets commis par erreur, **y compris dans l'historique** — effacer un secret dans un commit suivant ne l'efface pas. **nuclei** et un script maison attaquent l'application **en marche**. **Trivy** scanne les images : aucune faille corrigeable HIGH ou CRITICAL ne passe. Et **Syft** produit le SBOM, l'inventaire de ce qu'on livre.*

→ **Fonctionnalité 14 (sécurité DevSecOps)**

12 :50 — La chaîne, et ce qu'elle a trouvé (40 s)

```
conclusion : success
  success    SAST (bandit)
  success    Fuite de secrets (gitleaks)
  success    Lint Python (ruff)
  success    Lint Ansible
  success    Intégration (pile compose)
```

```
success   Images Docker (hadolint + Trivy)
success   Tests (pytest)
success   SCA (pip-audit)
success   DAST (application en fonctionnement)
skipped   Build, SBOM et publication
skipped   Déploiement (Ansible)
```

> *Onze travaux à chaque envoi. Et ils **bloquent** : aucune image n'atteint le registre si l'un échoue. Ce n'est pas décoratif — nos propres images ont été refusées par notre propre scan, et il a fallu changer de base.*
>
> *Dix défauts ont été trouvés en **faisant tourner** le projet, pas en le relisant : le Faucheur qui se supprimait lui-même, le mot de passe journalisé en clair, une course entre les quatre processus Gunicorn, une dépendance qui n'existait pas pour le Python de la cible. Chacun a maintenant son test ou son garde-fou — la chaîne vérifie même sa propre syntaxe shell avant de démarrer.*

→ Fonctionnalité 15 (CI/CD)

Transition : « [Prénom 1] conclut. »

CHAPITRE 4

Personne 1 — Conformité, ajouts, conclusion

13 :30 — La grille (50 s)

Les dix-sept fonctionnalités de l'énoncé sont traitées. Rapidement, l'outil employé pour chacune : Git et un environnement épinglé pour la préparation ; Flask et Gunicorn pour l'API ; SQLAlchemy et Alembic pour la base — sept tables, migrations versionnées ; Docker et supervisord pour la conteneurisation ; Vagrant pour l'infrastructure ; Ansible pour l'automatisation, neuf rôles idempotents ; un agent maison pour l'enregistrement ; un ordonnanceur qui prend le nœud joignable le moins chargé ; le Faucheur pour l'expiration et la reprise ; bandit, pip-audit, gitleaks, nuclei, Trivy et Syft pour la sécurité ; GitHub Actions pour la chaîne ; pytest pour les tests.

Et pour la documentation : le README, la doc de l'API, celle de la base, quatre schémas — et un ***guide technique de 93 pages*** *qui explique tout le projet depuis les notions de base.*

#	Fonctionnalité	Outil employé
1	Préparation, environnement	Git, environnement virtuel, dépendances épinglées
2	API Web Flask	Flask, Jinja2, Gunicorn (4 ouvriers), nginx
3	Base de données	SQLAlchemy 2.0, Alembic, PostgreSQL ou SQLite
4	Conteneurisation Docker	4 Dockerfiles, 6 images, supervisord, durcissement
5	Infrastructure Vagrant	4 VM, réseau privé, 4 providers
6	Automatisation Ansible	9 rôles, 4 playbooks, Vault AES-256, idempotence
7	Worker Agent	agent stdlib, inscription + battement de cœur
8	Ordonnanceur	nœud joignable le moins chargé, départage déterministe
9	Service de location	1 à 120 min, prolongation, arrêt
10	Déploiement des instances	`subprocess` + la commande imposée
11	Accès SSH	mot de passe ou clé, + ttyd + noVNC
12	Expiration	le Faucheur, toutes les 10 s, avec réconciliation
13	Haute disponibilité	battement de cœur, `OFFLINE`, recréation ailleurs
14	Sécurité DevSecOps	bandit, pip-audit, gitleaks, nuclei, Trivy, Syft, Fernet
15	CI/CD	GitHub Actions, 11 travaux, ghcr.io, SBOM
16	Tests	147 unitaires + 6 scénarios de bout en bout
17	Documentation	README, API, base, 4 schémas, guide de 93 pages

→ Fonctionnalité 1 et 17

14 :20 — Ce qu'on a ajouté (30 s)

> *Sept choses au-delà de ce qui était demandé : un parc **distribué à trois workers** alors qu'un seul hôte suffisait ; le **terminal web** et le **bureau graphique** en plus du SSH ; un **coffre à secrets** qui réaffiche le mot de passe après re-authentification ; un **watchdog** systemd qui répare le service tout seul ; un **inventaire dynamique** qui interroge Vagrant, ce qui rend le déploiement indépendant du provider ; la **sauvegarde du travail** avant destruction ; et des **schémas engendrés depuis leur description**, que la chaîne régénère et refuse de laisser vieillir.*

14 :50 — Fermer (10 s)

> *Le projet tourne, il est testé, et il se redéploie en une commande. Nous sommes à votre disposition pour les questions.*

CHAPITRE 5

Les questions probables

Qui	Question	Réponse
P1	Pourquoi des conteneurs et pas des VM ?	Démarrage sous la seconde, pas de noyau invité à charger. Et l'énoncé le tranche : « des conteneurs dans notre cas ».
P1	Pourquoi SSH plutôt que l'API Docker ?	Qui parle au démon Docker d'une machine en est root. Avec `docker -H ssh://`, aucun port Docker n'est ouvert.
P2	Où est stocké le mot de passe root ?	Chiffré avec Fernet, clé distincte de celle des sessions, rangée dans le Vault. Jamais dans un cookie — une session Flask est signée, pas chiffrée.
P2	Et si deux utilisateurs demandent le même port ?	Index unique partiel en base, limité aux machines actives : le port redevient libre dès que la machine est détruite.
P2	Pourquoi supervisord dans le conteneur ?	Un conteneur exécute un processus ; il en faut cinq en mode bureau. Et le PID 1 doit adopter les orphelins, ce qu'un shell ne fait pas.
P3	La reprise rend-elle les fichiers ?	Non, et c'est assumé. Ils vivaient dans la couche inscriptible du conteneur, sur un nœud injoignable. Sans stockage partagé, seul le service repart.
P3	Comment ajoute-t-on un worker ?	On l'allume. Aucune adresse n'est écrite côté contrôleur : l'agent s'inscrit tout seul.
P3	Pourquoi 162 tests sans Docker ? [22]	`run_docker` est remplacé par un espion : on vérifie que la commande imposée part bien, sur un agent qui n'a pas Docker.
P1	Combien de temps pour tout remonter ?	`vagrant up` puis un

CHAPITRE 6

Si quelque chose casse en direct

> **À retenir**
> Une panne commentée vaut mieux qu'un silence. Si quelque chose ne marche pas, on dit à voix haute ce qu'on regarde et pourquoi — c'est aussi une démonstration de maîtrise.

Symptôme	Quoi faire, à voix haute
Le site ne répond pas	`docker compose -p insacloud ps`, puis `logs web --tail 20`. Commenter ce qu'on lit.
Le mode bureau n'apparaît pas	Les images `*_desktop` ne sont pas construites — et c'est volontaire : le catalogue ne propose pas ce qu'il ne peut pas livrer.
`port is already allocated`	Un conteneur d'un essai précédent traîne : `docker ps -a --filter "label=insacloud.role=rented-machine"`.
La machine met du temps à répondre en SSH	En parler : Docker publie le port avant que `sshd` écoute. C'est pour ça que nos tests réessaient au lieu d'attendre une durée fixe.
Le Faucheur n'a pas détruit la machine	Il passe toutes les dix secondes. Montrer `docker logs insacloud-faucheur` et attendre le cycle.

Ce qu'il ne faut pas faire

— **Ne pas lire ce script.** Le relire trois fois, puis parler.
— **Ne pas enchaîner les commandes en silence.** Pendant que ça charge, on explique.
— **Ne pas promettre ce qu'on ne montrera pas.** Si le cluster n'est pas monté, le dire au moment du Test 6, avec la mesure à l'appui.
— **Ne pas dépasser.** En cas de retard, Personne 3 coupe les six scénarios d'intégration (11 :30) et garde le Test 6 — c'est lui qui compte.

Dépôt : https://github.com/aziz3r/insacloud
Les captures de ce document viennent de la plateforme telle qu'elle tourne, et les sorties reproduites sont les vraies.